

NOTICE

- Deadlines
 - (Overdue) Programming assignment 2
 - (5/1 11:59 PM) Midterm Quiz 2
 - (5/13 11:59 PM) Programming assignment 3

CS 374: OPERATING SYSTEMS I

PART III – SIGNALS AND PIPES

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University

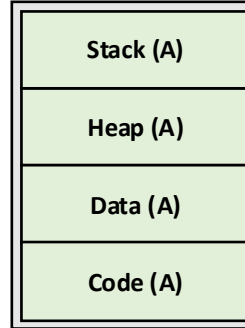


TRUE AI
Trustworthy and Responsible AI

RECAP: PROCESS ISOLATION

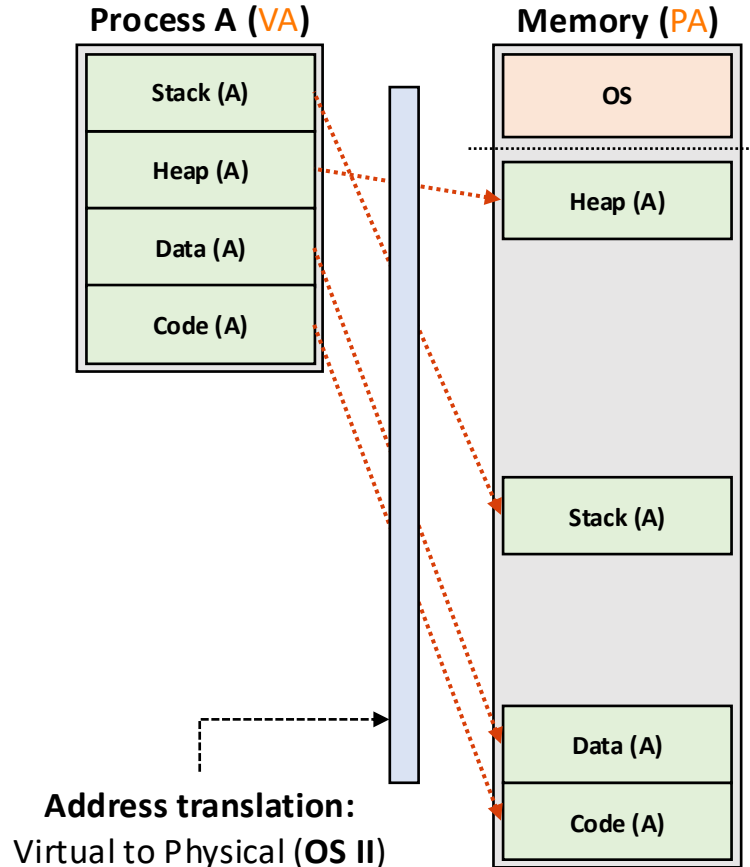
- Process segments
 - Code segment
 - Data segment
 - Heap segment
 - Stack segment

Process A (VA)



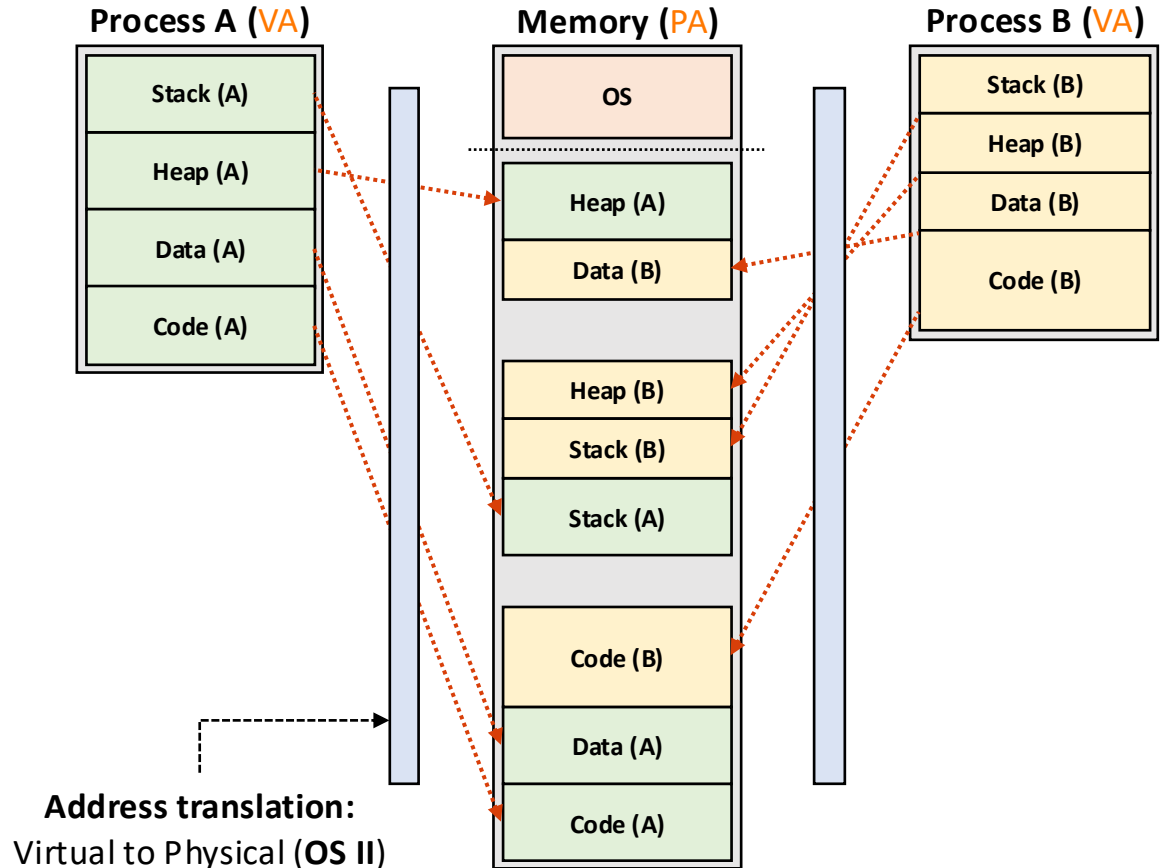
RECAP: PROCESS ISOLATION

- Process segments
 - Code segment
 - Data segment
 - Heap segment
 - Stack segment



RECAP: PROCESS ISOLATION

- Process segments
 - Code segment
 - Data segment
 - Heap segment
 - Stack segment



STRAWMAN SOLUTIONS

- **Hole punching ([Link](#))!**

- **Definition:**

- (from computer networking)
 - A technique that allows two or more parties to communicate directly each other

- **Downside:**

- Potentially ignore the security mechanisms (e.g., firewalls)
 - Potentially increase overheads to manage such connections separately
 - ...

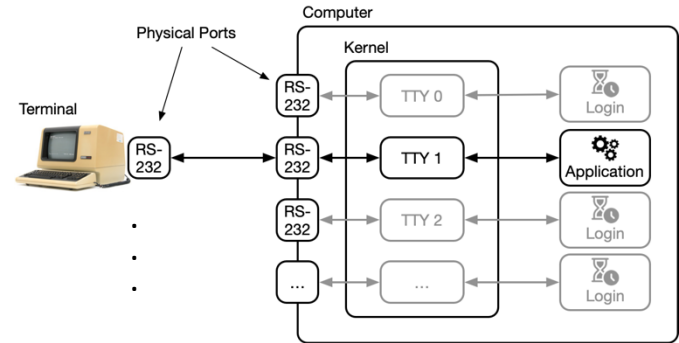
TOPICS FOR TODAY

- Part III: IPC, RPC, and Networking
 - Provide abstractions
 - What is IPC/RPC?
 - Why do we need IPC/RPC?
 - What is the mechanisms OS support for IPC?
 - Offer standard interface
 - What IPC primitives does the OS offer (signals, pipes, sockets)
 - How do we use them?
 - Manage resources
 - (Overview) How does the OS manage these resources?

PROVIDE ABSTRACTION

- **Background** (in 1960-70s)

- Terminals are connected to a (huge) computer
- You use terminal to control multiple processes
- You want to kill a process; how would you do?



- **OS support “signals”**

- **Definition:**

- (Formal) an asynchronous mechanism to notify an event to a process
- (Informal) **notifications** between processes or a process and a thread

PROVIDE ABSTRACTION

- **Signals**

- **Definition:**

- (Formal) an asynchronous mechanism to notify an event to a process
 - (Informal) **notifications** between processes or a process and a thread

- **Examples:**

- Hardware exception: SIGSEGV (illegal mem access), SIGFPE (divide by 0), ...
 - Terminal input: SIGINT (Ctrl + C), SIGTSTP (Ctrl + Z), SIGQUIT (Ctrl + \), ...
 - Between processes: kill(pid, sig), raise(sig)
 - OS Kernel: SIGCHLD, SIGALRM, SIGPIPE

kill -l // all signals available
man 7 signal // detailed descriptions

PROVIDE ABSTRACTION

- Signals in Linux

- 32 non-real-time signals (0 to 31)
- 31 real-time signals (32 to _NSIG [\[link\]](#))

#	Signal Name	Default Action	Comment	POSIX
1	SIGHUP	Terminate	Hang up controlling terminal or process	Yes
2	SIGINT	Terminate	Interrupt from keyboard, Control-C	Yes
3	SIGQUIT	Dump	Quit from keyboard, Control-\	Yes
4	SIGILL	Dump	Illegal instruction	Yes
5	SIGTRAP	Dump	Breakpoint for debugging	No
6	SIGABRT	Dump	Abnormal termination	Yes
6	SIGIOT	Dump	Equivalent to SIGABRT	No
7	SIGBUS	Dump	Bus error	No
8	SIGFPE	Dump	Floating-point exception	Yes
9	SIGKILL	Terminate	Forced-process termination	Yes
10	SIGUSR1	Terminate	Available to processes	Yes
11	SIGSEGV	Dump	Invalid memory reference	Yes
12	SIGUSR2	Terminate	Available to processes	Yes
13	SIGPIPE	Terminate	Write to pipe with no readers	Yes
14	SIGALRM	Terminate	Real-timer clock	Yes
15	SIGTERM	Terminate	Process termination	Yes
16	SIGSTKFLT	Terminate	Coprocessor stack error	No
17	SIGCHLD	Ignore	Child process stopped or terminated or got a signal if traced	Yes
18	SIGCONT	Continue	Resume execution, if stopped	Yes
19	SIGSTOP	Stop	Stop process execution, Ctrl-Z	Yes
20	SIGTSTP	Stop	Stop process issued from tty	Yes
21	SIGTTIN	Stop	Background process requires input	Yes
22	SIGTTOU	Stop	Background process requires output	Yes
23	SIGURG	Ignore	Urgent condition on socket	No
24	SIGXCPU	Dump	CPU time limit exceeded	No
25	SIGXFSZ	Dump	File size limit exceeded	No
26	SIGVTALRM	Terminate	Virtual timer clock	No
27	SIGPROF	Terminate	Profile timer clock	No
28	SIGWINCH	Ignore	Window resizing	No
29	SIGIO	Terminate	I/O now possible	No
29	SIGPOLL	Terminate	Equivalent to SIGIO	No
30	SIGPWR	Terminate	Power supply failure	No
31	SIGSYS	Dump	Bad system call	No
31	SIGUNUSED	Dump	Equivalent to SIGSYS	No

PROVIDE ABSTRACTION

- **Signal lifecycle**

- Each process has 3 signal bitmasks (in the `task_struct`)
 - Pending `[0 1 0 0 0 1 0 0 0 0 0 0 0 0 ...]` `// 1 – generated, 0 – no signal`
 - Blocked `[0 0 0 0 0 0 0 1 0 0 0 0 0 0 ...]` `// 1 – masked (block), 0 – allowed`
 - Handlers `[ SIG_DFL, my_fn, SIG_IGN, ...]` `// functions to be called upon delivery`
- Procedure
 - Generation : a signal has been created, `process.pending |= (1 << SIGINT)`
 - Pending : the signal sits in pending, `process.blocked & (1 << sig)`
 - Delivery check: OS kernel hands control back to a process
 - `process.pending & ~process.blocked`
 - If `deliver == 0`: return to the user code
 - If `deliver != 0`: pick one signal, clear its pending bit, and deliver it
 - Delivery : `SIG_DFL` (term, stop, ...), `SIG_IGN` (ignore), `my_fn` (run my function)

PROVIDE ABSTRACTION

- **Signal lifecycle (OS-level)**

- Process A sends a signal to Process B
- OS kernel updates B's process context (**Send**)
- OS kernel asks B to react to the signal (**Receive**)
 - Process B will execute a signal handler
 - Process B declines to receive the signal
- Multiple processes send signals to B (**Pending**)
 - Up to 1 pending signal per type for each process
 - More signals of the same type will be discarded

```
1085  /* Signal handlers: */
... 1086  struct signal_struct      *signal;
1087  struct sighand_struct __rcu *sighand;
1088  sigset_t                  blocked;
1089  sigset_t                  real_blocked;
1090  /* Restored if set_restore_sigmask() was used: */
1091  sigset_t                  saved_sigmask;
1092  struct sigpending          pending;
1093  unsigned long              sas_ss_sp;
1094  size_t                    sas_ss_size;
1095  unsigned int               sas_ss_flags;
1096
1097  struct callback_head       *task_works;
1098
1099  #ifdef CONFIG_AUDIT
1100  #ifdef CONFIG_AUDITSYSCALL
1101      struct audit_context    *audit_context;
1102  #endif
1103      kuid_t                  loginuid;
1104      unsigned int            sessionid;
1105  #endif
1106  struct seccomp              seccomp;
1107  struct syscall_user_dispatch syscall_dispatch;
1108  ...
```

PROVIDE ABSTRACTION

- Delivery from Process A to B
 - **OS kernel**
 - Checks if Process B has pending signals
 - Pauses the execution of Process B
 - Invokes `do_signal()`
 - `do_signal()` call invokes `handle_signal()`
 - **Process B**
 - Run code in `signal_handler`
 - Return back to kernel: `sigreturn()`
 - **OS Kernel**
 - Resume Process B

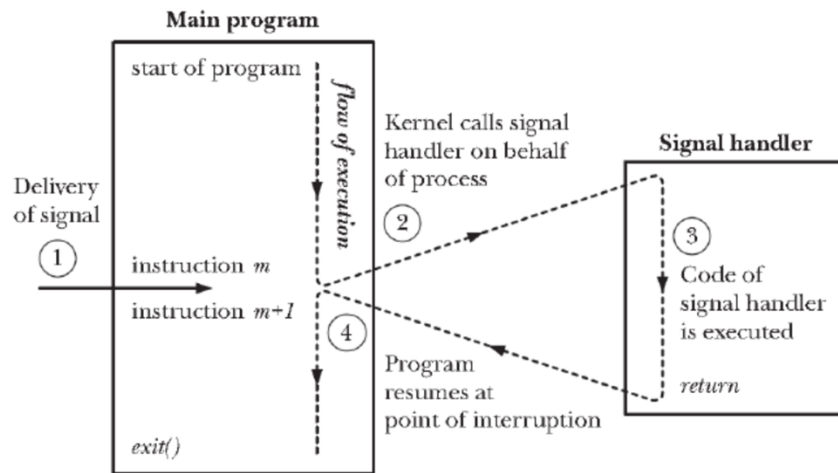


Figure 20-1: Signal delivery and handler execution

PROVIDE ABSTRACTION

- **Send signals**

- APIs

- `int kill(pid_t pid, int sig)` `// It sends any signal, not only SIGKILL!`
 - `int raise(int sig)`

- Detailed description of pid argument of kill()

- `pid > 0` The specific process with that PID
 - `pid == 0` Every process in the same process group
 - `pid == -1` Every process the caller has permission to signal
 - `pid < -1` Every process in the process group

PROVIDE ABSTRACTION

- Send signals – cont'd

- APIs

- `int kill(pid_t pid, int sig)` // It sends any signal, not only SIGKILL!
 - `int raise(int sig)`

... many #include

```
int main(void) {
    pid_t child = fork();
    if (child < 0) { perror("fork"); exit(1); }

    if (child == 0) {
        /* Child: loop indefinitely, waiting for a signal */
        printf("Child (PID=%d): sleeping, waiting for signal...\n", getpid());
        while (1) pause(); // blocks until any signal is delivered
        return 0;
    }

    /* Parent: wait 2 seconds, then send SIGTERM to the child */
    sleep(2);
    printf("Parent: sending SIGTERM to child (PID=%d)\n", child);
```

... (continue from the left)

```
    if (kill(child, SIGTERM) == -1) {
        perror("kill");
        exit(1);
    }

    /* Reap the child */
    int status;
    waitpid(child, &status, 0);

    if (WIFSIGNALED(status))
        printf("Child was terminated by signal %d\n", WTERMSIG(status));

    return 0;
}
```

PROVIDE ABSTRACTION

- Handling signals

- Legacy APIs

- `typedef void (*sighandler_t)(int);`
 - `sighandler_t signal(int sig, sighandler_t handler);`

... many #include

```
/*
 * volatile sig_atomic_t: the only integer type guaranteed to be
 * readable and writable atomically from a signal handler.
 */
volatile sig_atomic_t running = 1;

void sigint_handler(int sig) {
    /*
     * Only async-signal-safe functions may be called here.
     * printf() is NOT safe inside a signal handler — use write().
     */
    const char msg[] = "\nCaught SIGINT — shutting down...\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    running = 0;
}
```

... (continue from the left)

```
int main(void) {
    if (signal(SIGINT, sigint_handler) == SIG_ERR) {
        perror("signal");
        return 1;
    }

    printf("Running... press Ctrl+C to stop\n");
    while (running) {
        sleep(1);
        printf("tick\n");
    }

    printf("Exiting cleanly.\n");
    return 0;
}
```


PROVIDE ABSTRACTION

- **Handling signals**

- Modern APIs

- `int sigaction(int signum, construct struct sigaction *act, struct sigaction *oldact);`
 - `struct sigaction { handler, action, mask, flags };`

- Flags

- `SA_RESTART` : automatically restart interrupted system calls (read, write...)
 - `SA_SIGINFO` : use `sa_sigaction` that offers signal number, sender PID, and more...
 - `SA_NODEFER` : do not block this signal while its own handler runs
 - `SA_RESETHAND`: reset to `SIG_DFL` after one delivery

PROVIDE ABSTRACTION

- Handling signals

- Modern APIs

- `int sigaction(int signum, construct struct sigaction *act, struct sigaction *oldact);`
 - `struct sigaction { handler, action, mask, flags };`

... many `#include`

`volatile sig_atomic_t running = 1;`

```
void handler(int sig) {
    const char msg[] = "\nCaught signal — exiting\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    running = 0;
}
```

```
int main(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handler;
```

```
    /* While the handler runs, also block SIGTERM to avoid re-entry */
    sigemptyset(&sa.sa_mask);
    sigaddset(&sa.sa_mask, SIGTERM);
```

```
    /* Restart any syscalls that get interrupted by this signal */
    sa.sa_flags = SA_RESTART;
```

```
    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction"); return 1;
    }
```

```
    if (sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("sigaction"); return 1;
    }
```

```
    printf("Running... send SIGINT or SIGTERM to stop\n");
    while (running) {
        sleep(1); printf("tick\n");
    }
```

```
    printf("Clean exit.\n");
    return 0;
}
```

PROVIDE ABSTRACTION

- **Handling signals**

- Modern APIs

- `int sigaction(int signum, construct struct sigaction *act, struct sigaction *oldact);`
 - `struct sigaction { handler, action, mask, flags };`

- Mask signals: protect your *critical section*

- `int sigemptyset(sigset_t *set)` : initialize to empty set
 - `int sigfillset(sigset_t *set)` : add every signal
 - `int sigaddset(sigset_t *set, int sig)` : add one signal
 - `int sigdelset(sigset_t *set, int sig)` : remove one signal
 - `int sigprocmask(int how, const sigset_t *set, sigset_t *oldset)`
 - SIG_BLOCK (add)
 - SIG_UNBLOCK (remove)
 - SIG_SETMASK (replace)

PROVIDE ABSTRACTION

- Handling signals

- Mask signals example

```
... many #include
```

```
volatile sig_atomic_t flag = 0;
```

```
void handler(int sig) { flag = 1; }
```

```
int main(void) {
```

```
    struct sigaction sa = { .sa_handler = handler, .sa_flags = SA_RESTART };  
    sigemptyset(&sa.sa_mask);  
    sigaction(SIGINT, &sa, NULL);
```

```
    /* — Enter critical section: block SIGINT — */
```

```
    sigset_t block_set, old_set;  
    sigemptyset(&block_set);  
    sigaddset(&block_set, SIGINT);
```

```
    sigprocmask(SIG_BLOCK, &block_set, &old_set);
```

```
{  
    /*  
     * Ctrl+C pressed here? SIGINT is NOT delivered.  
     * Safe to modify shared state without interference.  
     */  
    printf("In critical section (SIGINT blocked)...\\n");  
    sleep(3);  
    printf("Critical section complete.\\n");  
}  
/* — Leave critical section: restore old mask — */  
sigprocmask(SIG_SETMASK, &old_set, NULL);  
  
/*  
 * If SIGINT was pending, it is delivered right here,  
 * and handler() runs before the next line executes.  
 */  
  
if (flag)  
    printf("SIGINT was received during critical section — now handled.\\n");  
  
return 0;  
}
```

PROVIDE ABSTRACTION

- **Signal safety**

- **Problems**

- A handler can run at any point in the program, even in `malloc()`
 - Those functions use internal mechanisms and data structures
 - Calling them from a handler can cause deadlock or heap corruption

- Async-signal safe functions

- `read`, `write`, `alarm`, `sleep`
 - `open`, `close`, `_exit`
 - `kill`, `getpid`, `waitpid`, `fork`
 - `signal`, `sigaction`, `sigpromask`

- Should *not* be called in handler

- `printf`, `fprintf`, `malloc`, `free`, `exit`

PROVIDE ABSTRACTION

- **Are we happy with signals?**
 - Our communication is limited to 31 types
 - We typically want to send more info (*e.g.*, filename to open)

PROVIDE ABSTRACTION

- **PIPE:**

- **Definition:** a unidirectional data channel, used for inter-process communication
- **Conceptually:**
 - A file shared between two process (only one can write, and the other can only read)
 - Note: a file descriptor can be shared between two process
 - **To write:** write(**writefd**, wbuf, wlen);
 - **To read :** read(**readfd**, rbuf, rmax);

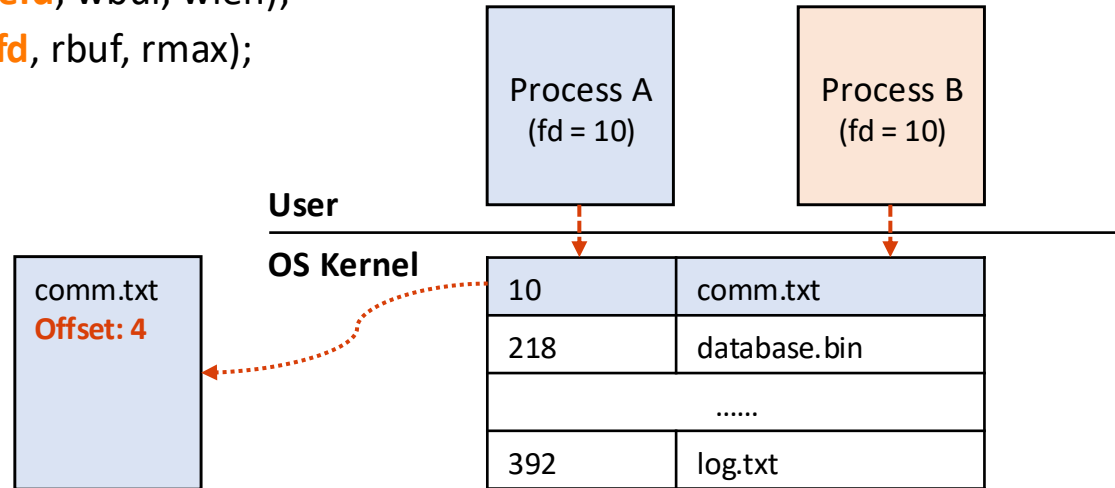
PROVIDE ABSTRACTION

- **PIPE:**

- **Definition:** a unidirectional data channel, used for inter-process communication
- **Conceptually:**
 - A file shared between two process (only one can write, and the other can only read)
 - Note: a file descriptor can be shared (**aliased**) between two process
 - **To write:** write(**writefd**, wbuf, wlen);
 - **To read :** read(**readfd**, rbuf, rmax);

- **Problem?**

- Too many storage access?



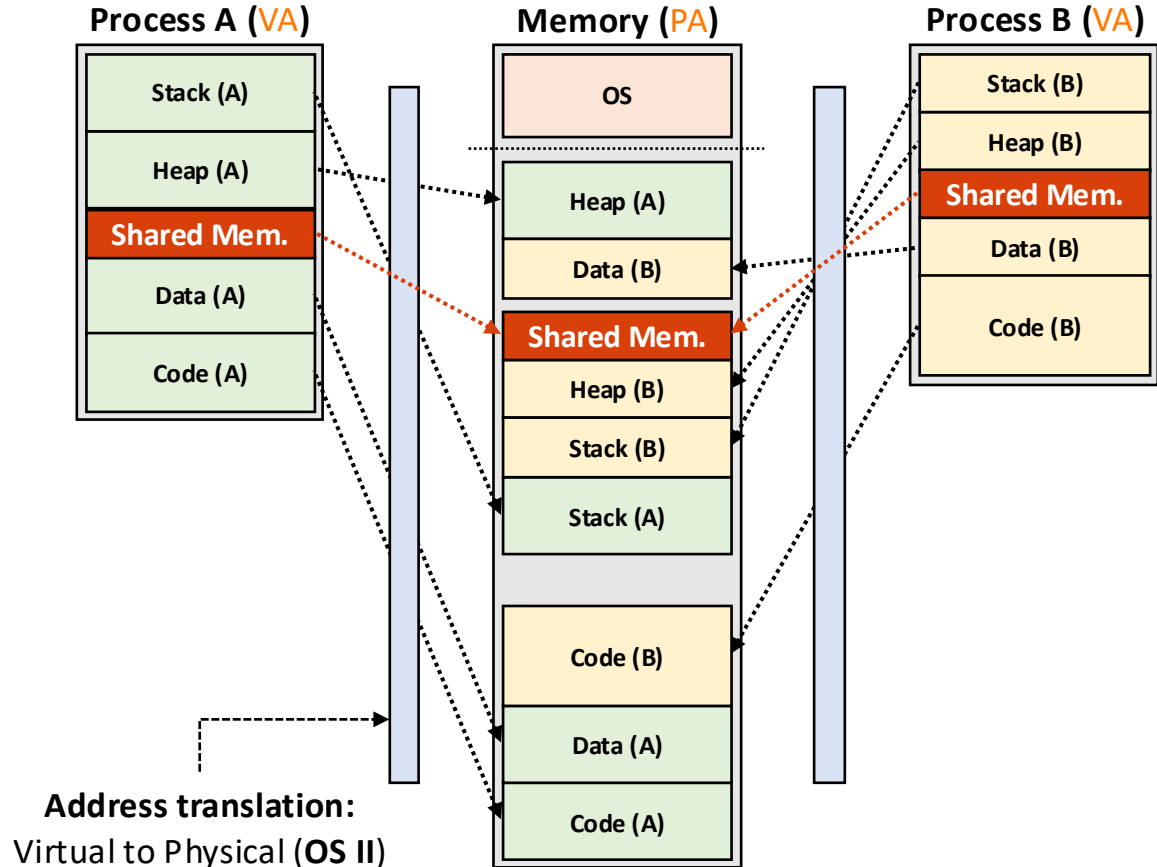
PROVIDE ABSTRACTION

- **Solution: memory!**
 - Disk access: 10^{-3}s
 - Mem. access: 10^{-9}s
 - Mem is $\sim 10^6\times$ faster

PROVIDE ABSTRACTION

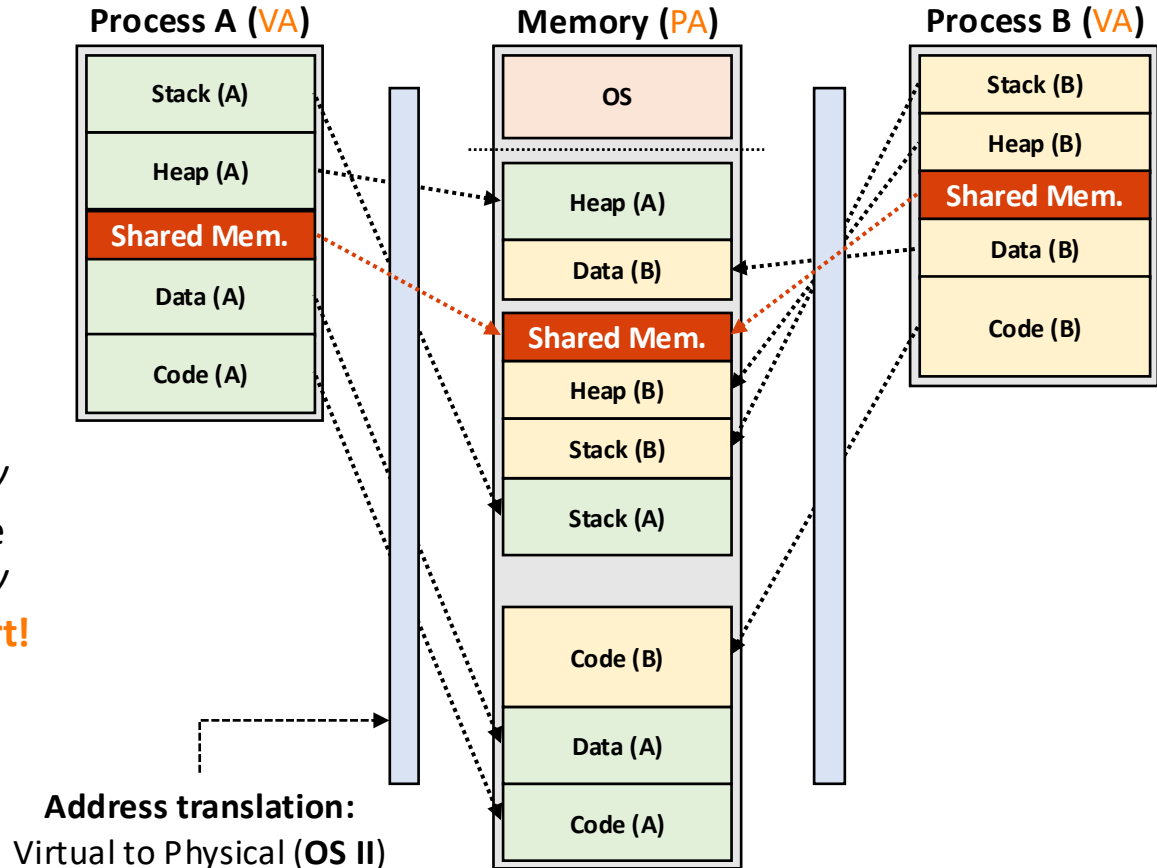
- **Solution: memory!**

- Disk access: $10^{-3}s$
- Mem. access: $10^{-9}s$
- Mem is $\sim 10^6x$ faster



PROVIDE ABSTRACTION

- **Solution: memory!**
 - Disk access: $10^{-3}s$
 - Mem. access: $10^{-9}s$
 - Mem is $\sim 10^6\times$ faster
- **Require OS support**
 - We should not allocate shared memory *arbitrarily*
 - We should not control the shared memory *arbitrarily*
 - **Require OS kernel support!**



OFFER STANDARD INTERFACE: PIPE

- System call for pipes
 - `int pipe( int fds[2] );`
 - It returns two file descriptors to “fds”
 - **fds[0]** is the fd for reading from the pipe
 - **fds[1]** is the fd for writing to the pipe
 - Note that the message size limit is 4096 bytes

OFFER STANDARD INTERFACE: PIPE

- System call for pipes
 - `int pipe( int fds[2] );`
 - It returns two file descriptors to “fds”
 - **`fds[0]`** is the fd for reading from the pipe
 - **`fds[1]`** is the fd for writing to the pipe
 - Note that the message size limit is 4096 bytes
- Tips to use “PIPEs” in Terminal
 - If you want to count the total number of files and directories: `ls | wc -l`
 - If you have many files for a screen: `ls -alh | more`
 - If you want to catch lines with a specific keywords: `cat <filename> | grep <keyword>`
 - If you want to remove the files with a prefix: `find ./ -name <prefix>* | xargs rm -f {} \;`
 - ...

OFFER STANDARD INTERFACE: PIPE

- **Basic PIPE example**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

#define BUFSIZE 512

int main(void) {
    char *msg = "It's a message in the pipe.";
    char buf[BUFSIZE];
    int pipe_fd[2];

    if (pipe(pipe_fd) == -1) {
        perror("Error, failed to open a pipe.\n");
        return EXIT_FAILURE;
    }

    ssize_t writelen = write(pipe_fd[1], msg, strlen(msg)+1);
    printf("Send: %s [%ld, %ld]\n", msg, strlen(msg)+1, writelen);
```

... (continue to the right)

... (continue from the left)

```
    ssize_t readlen = read(pipe_fd[0], buf, BUFSIZE);
    printf("Recv: %s [%ld, %ld]\n", buf, strlen(buf)+1, readlen);

    close(pipe_fd[0]);
    close(pipe_fd[1]);

    return 0;
}
```

OFFER STANDARD INTERFACE: PIPE

- **PIPE + fork() Pattern**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

#define BUFSIZE 512

int main(void) {
    char *msg = "It's a message in the pipe.\n";
    char buf[BUFSIZE];
    int pipe_fd[2];
    ssize_t readlen, writelen;

    if (pipe(pipe_fd) == -1) {
        perror("Error, failed to open a pipe.\n");
        return EXIT_FAILURE;
    }

    pid_t pid = fork();
```

... (continue to the right)

... (continue from the left)

```
if (pid < 0) {
    perror("Error, failed to fork().\n");
    return EXIT_FAILURE;
}

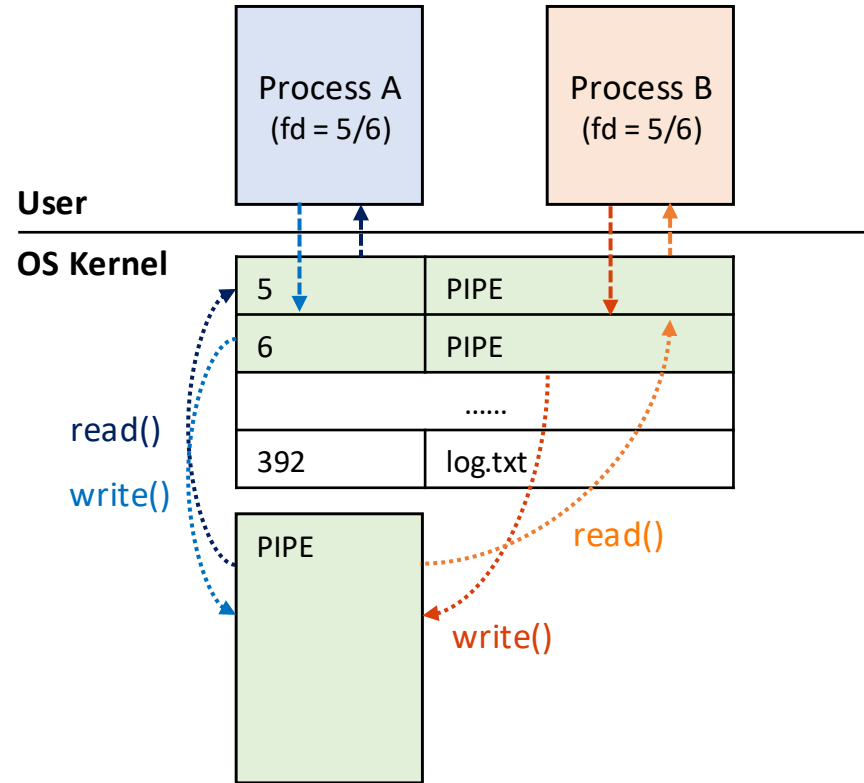
switch (pid) {
    case 0:
        readlen = read(pipe_fd[0], buf, BUFSIZE);
        printf("Recv: %s [%ld, %ld]\n", buf, strlen(buf)+1, readlen);
        close(pipe_fd[1]);
        break;

    default:
        writelen = write(pipe_fd[1], msg, strlen(msg)+1);
        printf("Send: %s [%ld, %ld]\n", msg, strlen(msg)+1, writelen);
        close(pipe_fd[0]);
}

return 0;
}
```

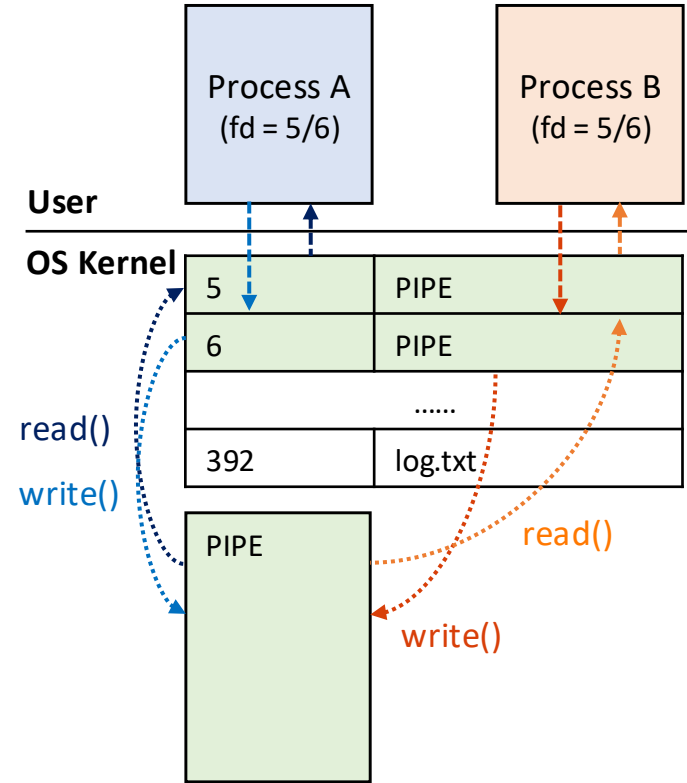
OFFER STANDARD INTERFACE: PIPE

- PIPE between two processes
 - Process A creates a pipe (fd=5/6)
 - A can read/write with the pipe
 - Process A fork()
 - Process B is created (a child)
 - Process B can read/write from (fd=5/6)



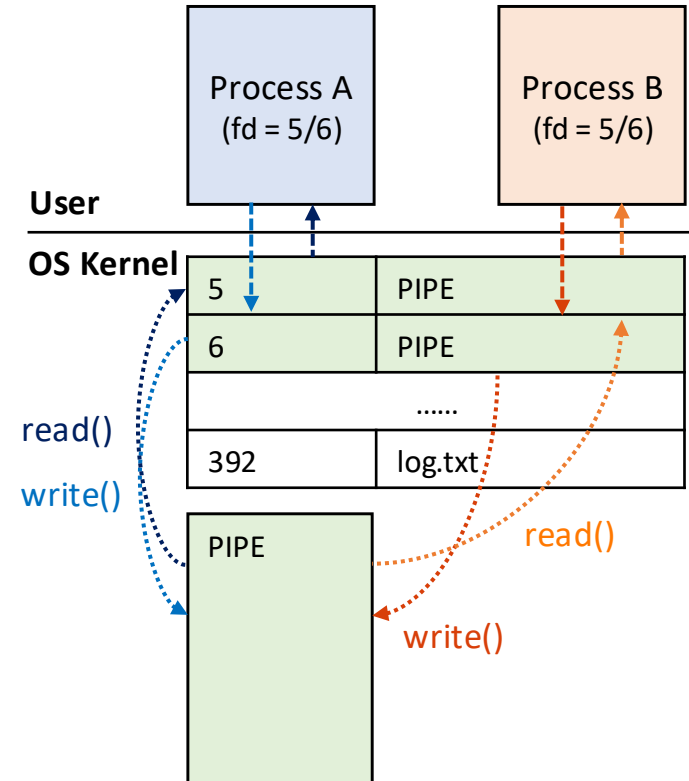
OFFER STANDARD INTERFACE: PIPE

- PIPE open/close
 - Process A closes “write” file descriptor
 - Process A can still read from the PIPE
 - Process B can still read/write to the PIPE



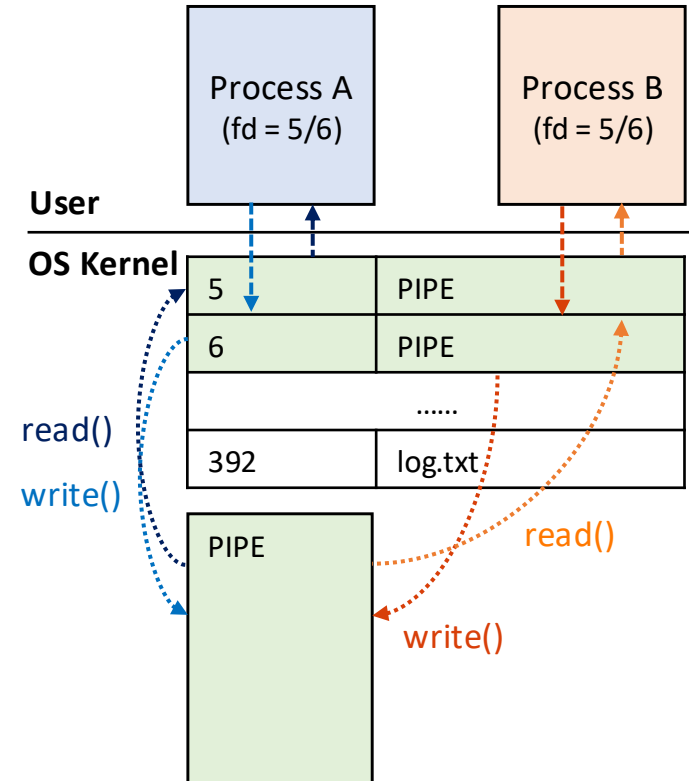
OFFER STANDARD INTERFACE: PIPE

- PIPE open/close
 - Process A closes “write” file descriptor
 - Process A can still read from the PIPE
 - Process B can still read/write to the PIPE
 - Process A and B close “write” file descriptors
 - Process A and B only read **EOF(0)** from the PIPE



OFFER STANDARD INTERFACE: PIPE

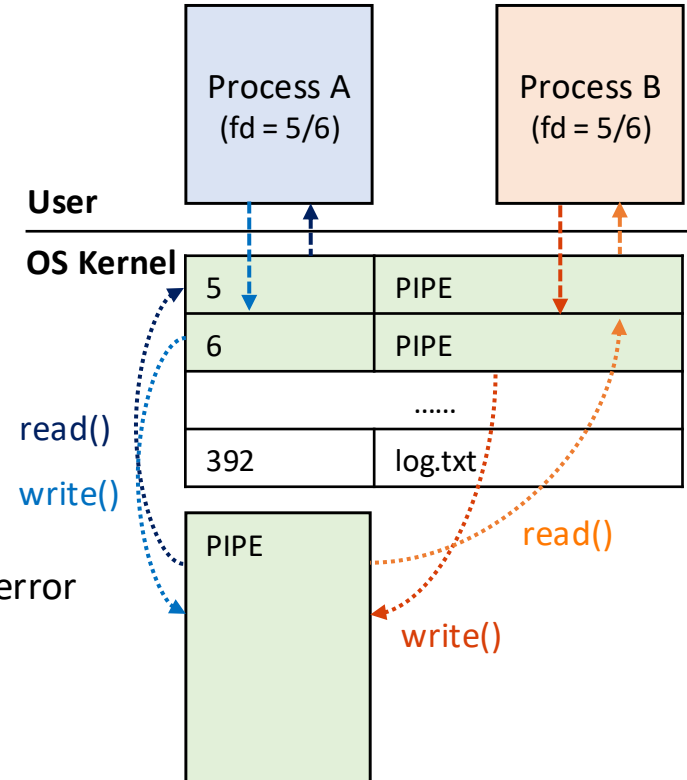
- PIPE open/close
 - Process A closes “write” file descriptor
 - Process A can still read from the PIPE
 - Process B can still read/write to the PIPE
 - Process A and B close “write” file descriptors
 - Process A and B only read EOF(0) from the PIPE
 - Process A closes “read” descriptors
 - Process A and B can write to the PIPE



OFFER STANDARD INTERFACE: PIPE

- PIPE open/close

- Process A closes “write” file descriptor
 - Process A can still read from the PIPE
 - Process B can still read/write to the PIPE
- Process A and B close “write” file descriptors
 - Process A and B only read EOF(0) from the PIPE
- Process A closes “read” descriptors
 - Process A and B can write to the PIPE
- Process A and B close “read” file descriptors
 - Process A or B’s “write” will fail and trigger **SIGPIPE** error
 - If **SIGPIPE** error is ignored, write will return **EPIPE**



OFFER STANDARD INTERFACE: PIPE

- **PIPE + exec(): dup2() and shell**
 - How `ls | grep sth` actually works?

```
int fd[2];
pipe(fd);

if (fork() == 0) {
    close(fd[0]);
    dup2(fd[1], STDOUT_FILENO); // replace stdout with pipe write
    close(fd[1]);
    execlp("ls", "ls", "-1", NULL);
    perror("exec"); exit(1);
} else {
    close(fd[1]);

    char buf[4096];
    ssize_t n = read(fd[0], buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("Output of ls:\n%s", buf);

    close(fd[0]);
    wait(NULL);
}
```

OFFER STANDARD INTERFACE: MKFIFO

- System call for pipes
 - `int mkfifo( const char *pathname, mode_t mode );`
 - mode is the permission bits, e.g., 0644
 - **Returns 0** on success
 - **Returns -1** on error (EEXIST if already exists)
 - `$ mkfifo /tmp/myfifo`
 - Behavior
 - `open(path, O_RDONLY)` : blocks until another process opens the write
 - `open(path, O_WRONLY)` : blocks until another process opens the read end
 - add `O_NOBLOCK` to avoid blocking on open
 - No data will be written to disk (it lives only in the kernel buffer)

OFFER STANDARD INTERFACE: MKFIFO

- **Named PIPE read/write**

... many #include

```
#define FIFO_PATH "/tmp/demo_fifo"
```

```
int main(void) {
    if (mkfifo(FIFO_PATH, 0644) == -1)
        perror("mkfifo"); // EEXIST is fine — the FIFO is already there

    printf("Writer: opening FIFO (blocking until reader opens)...\n");
    int fd = open(FIFO_PATH, O_WRONLY);
    if (fd == -1) {
        perror("open"); exit(1);
    }

    const char *msg = "Hello from the writer process!\n";
    write(fd, msg, strlen(msg));
    printf("Writer: message sent, closing.\n");

    close(fd);
    unlink(FIFO_PATH); // remove the FIFO from the filesystem
    return 0;
}
```

... many #include

```
#define FIFO_PATH "/tmp/demo_fifo"
```

```
int main(void) {
    printf("Reader: opening FIFO (blocking until writer opens)...\n");
    int fd = open(FIFO_PATH, O_RDONLY);
    if (fd == -1) {
        perror("open"); exit(1);
    }

    char buf[256] = {0};
    ssize_t n = read(fd, buf, sizeof(buf) - 1);
    printf("Reader: received %zd bytes: \"%s\"\n", n, buf);

    close(fd);
    return 0;
}
```

OFFER STANDARD INTERFACE: PIPE AND MKFIFO

- **Common mistakes**

- Not closing unused pipe ends : `read()` never gets EOF, blocks forever
- Treating pipes as bidirectional : a pipe is one-way; must use two pipes
- Ignoring SIGPIPE : writing to a pipe with read end is closed; terminates
- Not the entire buffer will be transferred in a single call
- Not calling `unlink()` on a FIFO : the filesystem entry persists after the program ends

TOPICS FOR TODAY

- Part III: IPC, RPC, and Networking
 - Provide abstractions
 - What is IPC/RPC?
 - Why do we need IPC/RPC?
 - What is the mechanisms OS support for IPC?
 - Offer standard interface
 - What IPC primitives does the OS offer (signals, pipes, sockets)
 - How do we use them?
 - Manage resources
 - (Overview) How does the OS manage these resources?

Thank You!

M/W 12:00 – 1:50 PM (LINC #200)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI